Chapter 1. Introduction

Instructor: Kuen-Liang Sue
IM/NCU

Outline

- 1-1 Pseudocode
- 1-2 The Abstract Data Type
- 1-3 A Model for an Abstract Data Type
- 1-4 Algorithm Efficiency

1-1 Pseudocode

- Pseudocode
  - An English-like representation of the code
  - Part English
  - Part structured code
    - consists of sequence, selection, and iteration

One of the most common tools for defining algorithms is pseudocode, which is part English, part structured code.

Pseudocode

- Basic format for data type
  - Name of the data
  - Type enclosed in pointed brackets

```
count <integer>
```

- Structure of the data

```
node
    data    <dataType>
    link    <pointer to node>
end node
```

  - Assuming that dataType has been previous defined

Pseudocode

- Algorithm header
  - Each algorithm begins with a header
    - that names it, describes its parameters, and lists any pre- and postconditions
  - The programmer using the algorithm often see only the header, not the complete algorithm
    - The header information must be complete enough to communicate to the programmer

Pseudocode

- Example of Header

```
algorithm sample (ref pageNumber <integer>)
This algorithm reads a file and prints a report.
   Pre    pageNumber must be initialized
   Post   Report Printed. pageNumber contains number of pages
          in report.
   Return Number of lines printed.
```

Algorithm 1-1 Example of pseudocode

Example of pseudocode (cont.)

```
 1  open file
 2  lines = 0
 3  loop (not end of file)
    1   read file
    2   if (full page)
        1   form feed
        2   pageNumber = pageNumber + 1
        3   write page heading
    3   end if
    4   write report line
    5   lines = lines + 1
 4  end loop
 5  close file
 6  return lines
end sample
```

Algorithm 1-1 Example of pseudocode

Header in Pseudocode

- Purpose
  - A short statement about what the algorithm does
- Precondition
  - lists any precursor requirements for the parameters
  - Sometimes there are no preconditions, we still list it as shown below

```
Pre    Nothing
```

  - If there are several input parameters, then the precondition should be shown for each

Header in Pseudocode

- Example of array search algorithm

```
algorithm search  (val list      <array>,
                   val argument  <integer>,
                   ref location  <index>)
Search array for specific item and return index location.
    Pre     list contains data array to be searched
            argument contains data to be located in list
    Post    location contains index of element matching argument
            -or- undetermined if not found
    Return  <Boolean> true if found, false if not found
```


Header in Pseudocode

- Postcondition
  - identifies any action taken and the status of any output parameters
- Return condition
  - If a value is returned, it will be identified by a return condition
  - The example of search algorithm returns a Boolean
- Any Q?
  - What is difference between output parameter and return value?

Pseudocode

- Statement numbers
  - e.g. Algorithm 1-1
  - The expanded number of the statement that reads the files is 3.1
  - Statement 2 is an example of a sequence
  - Statement 3.2 is an example of a selection
  - Statement 3 is an example of a loop

Pseudocode

- Variables
  - Intelligent data names
    - names that describe the meaning of the data
  - Selection of the name
    - Do not use single character names
    - Do not use generic names
      - e.g. count, sum, total, row, column, and file
    - Abbreviations are not excluded as intelligent data names
      - e.g. numOfStu is a good abbreviation; noStu would not
- Algorithm analysis
  - Style or efficiency considerations

Statement constructs

- Statement constructs
  - Any algorithm could be written with only three programming constructs
    - Sequence
    - Selection
    - Loop
- Sequence
  - A series of statements
    - that do not alter the execution path within an algorithm

State constructs

- Selection
  - Selection statements evaluate one or more alternatives

```
1  if (condition)
   1  action1
2  else
   1  action2
3  end if
```

- Loop
  - iterates a block of code

Pseudocode example

```
algorithm deviation
   Pre    nothing
   Post   average and numbers with their deviation printed
 1 i = 0
 2 loop (not end of file)
    1   read number into array[i]
    2   sum = sum + number
    3   i = i + 1
 3 end loop
 4 average =  sum / i
 5 print (average)
 6 j = 0
 7 loop (j < i)
    1   devFromAve = array[j] - average
    2   print (array[j], devFromAve)
    3   j = j + 1
 8 end loop
 9 return
end deviation
```

Algorithm 1-2 Print deviation from mean for series of numbers

Pseudocode example

- Algorithm 1-2 analysis
  - There are no parameters
    - If there were , we would have indicated whether they were pass by reference (ref) or pass by value (val), their name, and their type
  - Using a code-oriented pseudocode as opposed to a "tight English" pseudocode
    - *Add one to i*
  - Pseudocode arrays are indexed starting at 0

1-2 The Abstract Data Type

- Spaghetti code
  - Nonstructural, linear programs
- Modular programming
  - Programs were organized in functions
    - each of which still used a linear coding technique
  - The structured programming concepts we still use today were formulated in the 1970s

The Abstract Data Type

- Data type
  - consists of two parts, a set of data and the operations that can be performed on the data
- Object-oriented programming
  - The functions are developed around an object
    - such as a linked list
  - Encapsulation
    - One part of the object-oriented concept
    - All processing for an object is bundled together in a library and hidden from user

The Abstract Data Type

- Atomic data
  - Data that we choose to consider as a single, nondecomposable entity
  - Atomic data type

Atomic Data Type

1. A set of values
2. A set of operations on values

The Abstract Data Type

- e.g. three atomic data types

integer
values: -∞, . . . , -2, -1, 0, 1, 2, . . . , ∞
operations: *, +, -, %, /, ++, --, . . .

floating point
values: -∞, . . . , 0.0, . . . , ∞
operations: *, +, -, /, . . .

character
values: \0, . . . , 'A', 'B', . . . , 'a', 'b', . . . , ~
operations[1]: <, >, . . .

- Composite data
  - The opposite of atomic data
  - e.g. telephone number has three different part
    - Area code, three-digit exchange and the number within the exchange

The Abstract Data Type

- Data Structure
  - An aggregation of atomic and composite data types into a set with defined relationships

Data Structure

1. A combination of elements each of which is either a data type or another data structure
2. A set of associations or relationships (structure) involving the combined elements

Array	Record
Homogeneous sequence of data or data types known as elements	Heterogeneous combination of data into a single structure with an identified key
Position association among the elements	No association

Table 1-1 Data structure examples

The Abstract Data Type

- Abstract Data Type
  - Users are not concerned with how the task is done but rather with what it can do
  - ADT consists of a set of definitions that allow programmers to use the functions while hiding the implementation
  - Abstraction

The concept of abstraction means:

1. We know *what* a data type can do.
2. *How* it is done is hidden.

The Abstract Data Type

- If we place our list in an ADT
  - user should not be aware of the structure we use
  - Four logical structures that might be used to hold a list

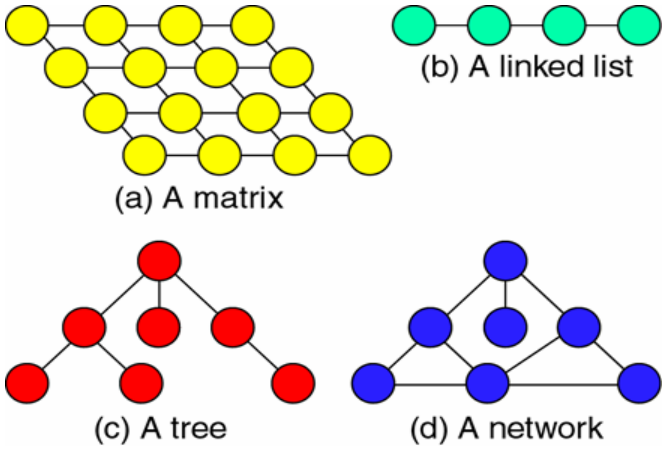


Figure 1-1 Some data structures

Example of an ADT

- Consider a system analyst who needs to simulate the waiting line of a bank
  - This analysis requires the simulation of a queue
    - However, queues are not generally available in programming languages
    - Even if a queue type were available, the analyst would still need some basic queue operations
      - Enqueuing and dequeuing

Example of an ADT

- Two potential solutions to this problem
  - Writing a program that simulates the needed queue
    - The solution is good only for one application at hand
  - Writing a queue ADT that can be used to solve any queue problem
- If choosing the latter course
  - writing a program to simulate the banking application is still needed
  - but he can concentrate on the application rather than queue

The Abstract Data Type

- Formal definition of an ADT
  - A data declaration packaged together with the operations that are meaningful for the data type

Abstract Data Type

1. Declaration of data
2. Declaration of operations
3. Encapsulation of data and operations

1-3 A Model for an Abstract Data Type

- ADT Model
  - e.g. Shown in Figure 1-2
  - Shaded area with irregular outline represent the model
  - Inside the area are two different aspects of the model
    - the data structure and the operational functions
    - Both are entirely contained in the model
    - Both are not within the user's scope

A Model for an Abstract Data Type

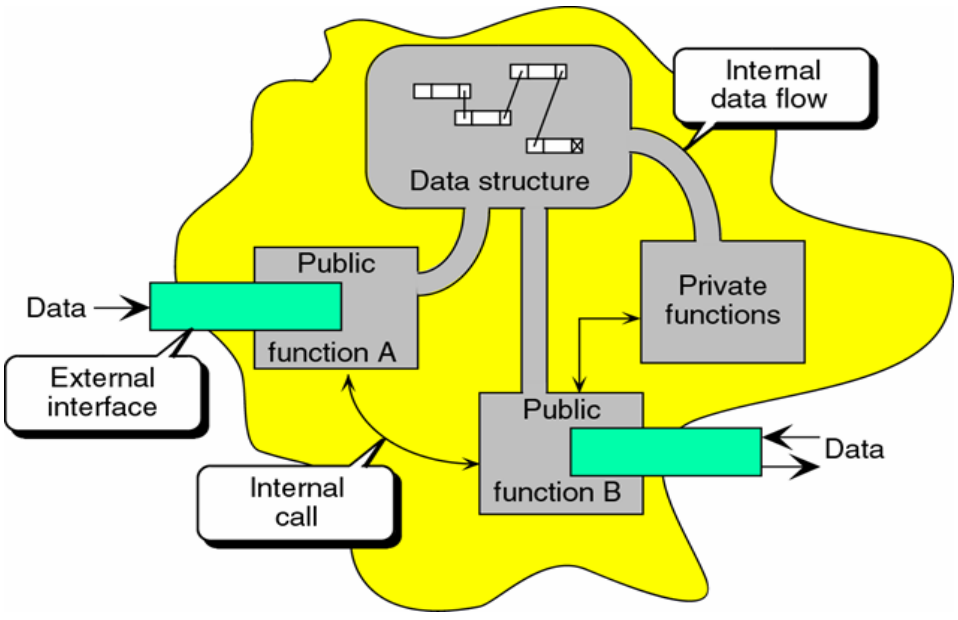


Figure 1-2 Abstract data type model

A Model for an Abstract Data Type

- Data operations
  - Data are entered, accessed, modified, and deleted through the operational interfaces
  - Only the operation name and its parameter are visible to the user
- ADT data structure
  - We must hide the implementation from the user while being able to store different data
  - Each ADT class object will have a defined type that users must use in their programs, e.g. file

A Model for an Abstract Data Type

- A pointer to a hidden structure

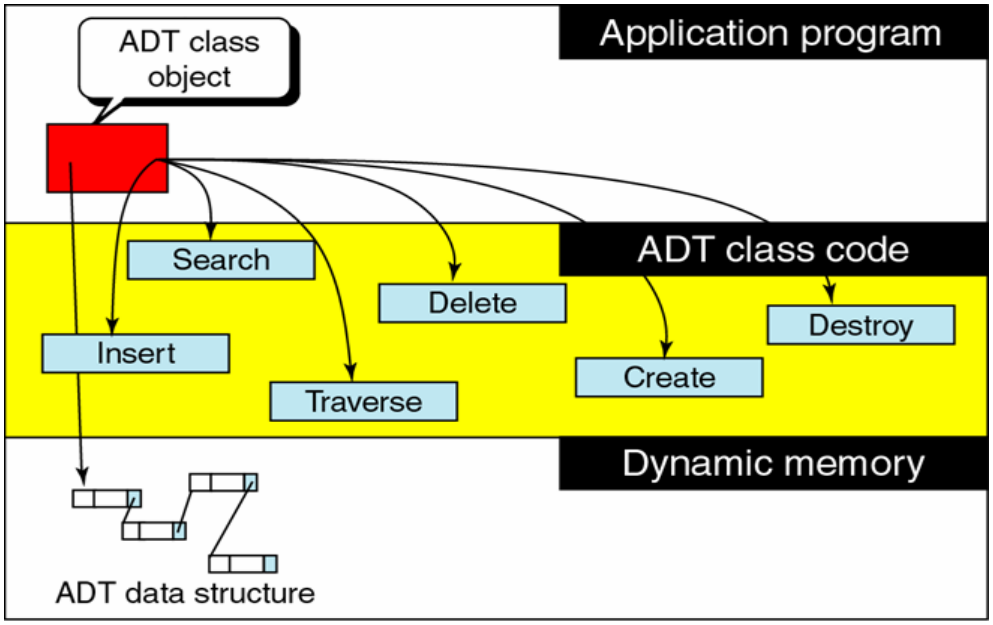


Figure 1-3 A pointer to a hidden structure

A Model for an Abstract Data Type

- ADT class templates
  - Two general components of a structure are
    - Data
      - The data structure is given a template identifier of TYPE
    - Key identifier, optionally
      - A key type identifier, KTYPE
    - See Program 1-1
  - In Program 1-2, we create a structure, PICTURE
    - key is the year the picture won the Award
    - We define the list object (see Statement 11), designating that the structure type is PICTURE and the key type is short

ADT Class Templates

```
1  // Node Declaration
2     template <class  TYPE>
3        struct NODE
4           {
5            ...
6           } ; //  End of Node Declaration
7
8  // List Class Declaration
9     template <class  TYPE, class KTYPE>
10    class List
11       {
12        ...
13       } ; // End of class List Declaration
```

Program 1-1 Typical ADT class template components

```
1  struct PICTURE
2     {
3      short    key;
4      char     picture [STR_MAX];
5      char     director[STR_MAX];
6     } ; // PICTURE
7
8  int main (void)
9  {
10 // Local Definitions
11    List<PICTURE, short> list;
```

Program 1-2 ADT class template implementation

1-4 Algorithm Efficiency

- Algorithmics
  - Defined as “systematic study of fundamental techniques used to design and analyze efficiency algorithms”
  - The study of algorithm efficiency focuses on loops
  - We generally discuss the algorithm’s efficiency as a function of the number of elements to be processed

```
f(n) = efficiency
```


Algorithm Efficiency

- Linear loop

```
1  i = 1
2  loop (i <= 1000)
     1   application code
     2   i = i + 1
3  end loop
```

```
f(n) = n
```

```
1  i = 1
2  loop (i <= 1000)
     1   application code
     2   i = i + 2
3  end loop
```

```
f(n) = n / 2
```

Algorithm Efficiency

- Logarithmic loops
  - The controlling variable is multiplied or divided in each loop

Multiply Loops

```
1  i =  1
2  loop (i < 1000)
   1  application code
   2  i = i × 2
3  end loop
```

Divide Loops

```
1  i =  1000
2  loop (i >= 1)
   1  application code
   2  i = i / 2
3  end loop
```


Algorithm Efficiency

Multiply		Divide	
Iteration	Value of i	Iteration	Value of i
1	1	1	1000
2	2	2	500
3	4	3	250
4	8	4	125
5	16	5	62
6	32	6	31
7	64	7	15
8	128	8	7
9	256	9	3
10	512	10	1
(exit)	1024	(exit)	0

Table 1-2 Analysis of multiply and divide loops

```
multiply   2^Iterations < 1000
divide     1000 / 2^Iterations >= 1
```

$$f(n) = \lceil \log_2 n \rceil$$

Algorithm Efficiency

- Nested loops

```
Iterations = outer loop iterations × inner loop iterations
```

  - Linear logarithmic

```
1  i = 1
2  loop (i <= 10)
   1   j = 1
   2   loop (j <= 10)
       1   application code
       2   j = j * 2
   3   end loop
   4   i = i + 1
3  end loop
```

$10 \times \lceil \log_2 10 \rceil$

$f(n) = \lceil n \log_2 n \rceil$

Algorithm Efficiency

  - Dependent quadratic

```
1  i = 1
2  loop (i <= 10)
   1   j = 1
   2   loop (j <= i)
       1   application code
       2   j = j + 1
   3   end loop
   4   i = i + 1
3  end loop
```

$1 + 2 + 3 + \ldots + 9 + 10 = 55$

$\frac{(n+1)}{2}$

$f(n) = n\left(\frac{n+1}{2}\right)$

Algorithm Efficiency

- Quadratic

```
1  i = 1
2  loop (i <= 10)
   1   j = 1
   2   loop (j <= 10)
       1   application code
       2   j = j + 1
   3   end loop
   4   i = i + 1
3  end loop
```

- This formula generalizes to $f(n) = n^2$

Big-O Notation

- O(n)--on the order of n
  - If an algorithm is quadratic, we would say its efficiency is $O(n^2)$
- Big-O notation can be derived from *f*(n) using the following step
  - In each term, set the coefficient of the term to 1
  - Keep the largest term and discard the others

$$\log_2 n \quad n \quad n\log_2 n \quad n^2 \quad n^3 \ \ldots \ n^k \quad 2^n \quad n!$$

Algorithm Efficiency

- For example

$$f(n) = n\frac{(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

$$\downarrow$$

$$n^2 + n$$

$$\downarrow$$

$$n^2$$

$$\downarrow$$

$$O(f(n)) = O(n^2)$$

Algorithm Efficiency

- Consider another example

$$f(n) = a_j n^k + a_{j-1} n^{k-1} + \ldots + a_2 n^2 + a_1 n + a_0$$

$$\downarrow$$

$$f(n) = n^k + n^{k-1} + \ldots + n^2 + n + 1$$

$$\downarrow$$

$$O(f(n)) = O(n^k)$$

Algorithm Efficiency

- Standard measures of efficiency

Efficiency	Big-O	Iterations	Estimated Time[a]
Logarithmic	$O(\log_2 n)$	14	Microseconds
Linear	$O(n)$	10,000	0.1 seconds
Linear logarithmic	$O(n(\log_2 n))$	140,000	2 seconds
Quadratic	$O(n^2)$	$10{,}000^2$	15–20 minutes
Polynomial	$O(n^k)$	$10{,}000^k$	Hours
Exponential	$O(c^n)$	$2^{10{,}000}$	Intractable
Factorial	$O(n!)$	10,000!	Intractable

[a]Assumes instruction speed of 1 microsecond and 10 instructions in loop.

Table 1-3 Measures of efficiency

Algorithm Efficiency

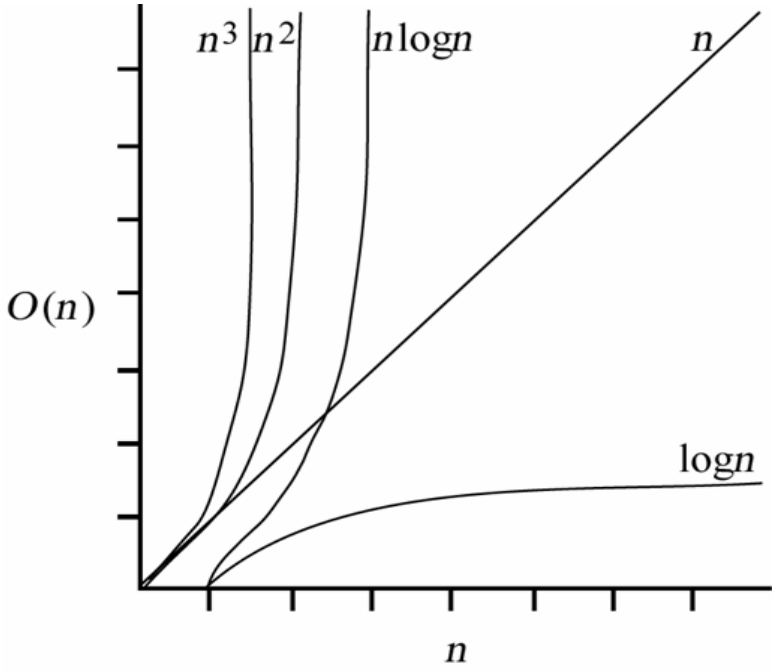


Figure 1-4 Big-O ranges

Algorithm Efficiency

- Big-O analysis example
  - Add matrices

4	2	1
0	–3	4
5	6	2

+

6	1	7
3	2	–1
4	6	2

=

10	3	8
3	–1	3
9	12	4

Figure 1-5 Add matrices

  - The efficiency of Algorithm 1-3 is $O(size^2)$ or $O(n^2)$

Algorithm Efficiency

```
algorithm addMatrix   (val matrix1   <matrix>,
                       val matrix2   <matrix>,
                       val size      <integer>
                       ref matrix3   <matrix>)
Add matrix1 to matrix2 and place results in matrix3
   Pre     matrix1 and matrix2 have data
           size is number of columns and rows in matrix
   Post    matrices added--result in matrix3
 1 r = 0
 2 loop (r < size)
   1   c = 0
   2   loop (c < size)
       1   matrix3[r, c] = matrix1[r, c] + matrix2[r, c]
       2   c = c + 1
   3   end loop
   4   r = r + 1
 3 end loop
 4 return
end addMatrix
```

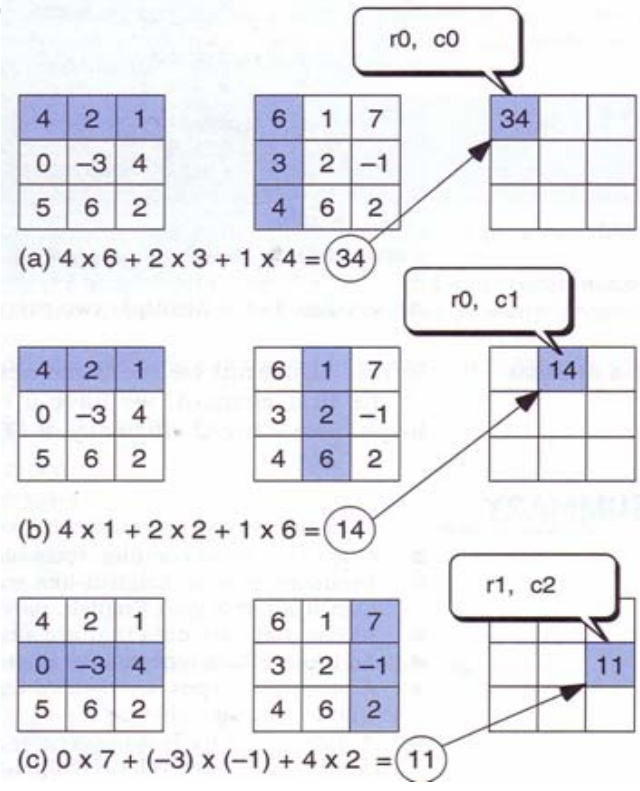
Algorithm 1-3 Add two matrices

Algorithm Efficiency

- Multiply matrices
  - When two matrices are multiplied, we must multiply each element in a row of the first matrix by its corresponding element in a column of the second matrix

```
matrix3 [r, c] =
        matrix1[r, 0] x matrix2[0, c]
    +   matrix1[r, 1] x matrix2[1, c]
    +   matrix1[r, 2] x matrix2[2, c]
    ...
    +   matrix1[r, s-1] x matrix2[s-1, c]
where s = size of matrix
```


Algorithm Efficiency



Algorithm Efficiency

- In this algorithm loops with three nested loops have a big-O efficiency of $O(size^3)$ or $O(n^3)$

```
Algorithm addMatrix   (val matrix1   <matrix>,
                       val matrix2   <matrix>,
                       val size      <integer>
                       ref matrix3   <matrix>)
Multiply matrix1 by matrix2 and place product in matrix3
    Pre    matrix1 and matrix2 have data
           size is number of columns and rows in matrix
    Post   matrices multiplied--result in matrix3
```

Algorithm 1-4 Multiply two matrices

Multiply Two Matrices (contd.)

```
 1  r = 0
 2  loop (r < size)
    1   c = 0
    2   loop (c < size)
        1   matrix3[r, c] = 0
        2   m = 0
        3   loop (m < size)
            1   matrix3[r, c]  = matrix3[r, c]
                               + matrix1[r, m] × matrix2[m, c]
            2   m = m + 1
        4   end loop
        5   c = c + 1
    3   end loop
    4   r = r + 1
 3  end loop
 4  return
end addMatrix
```

Algorithm 1-4 Multiply two matrices